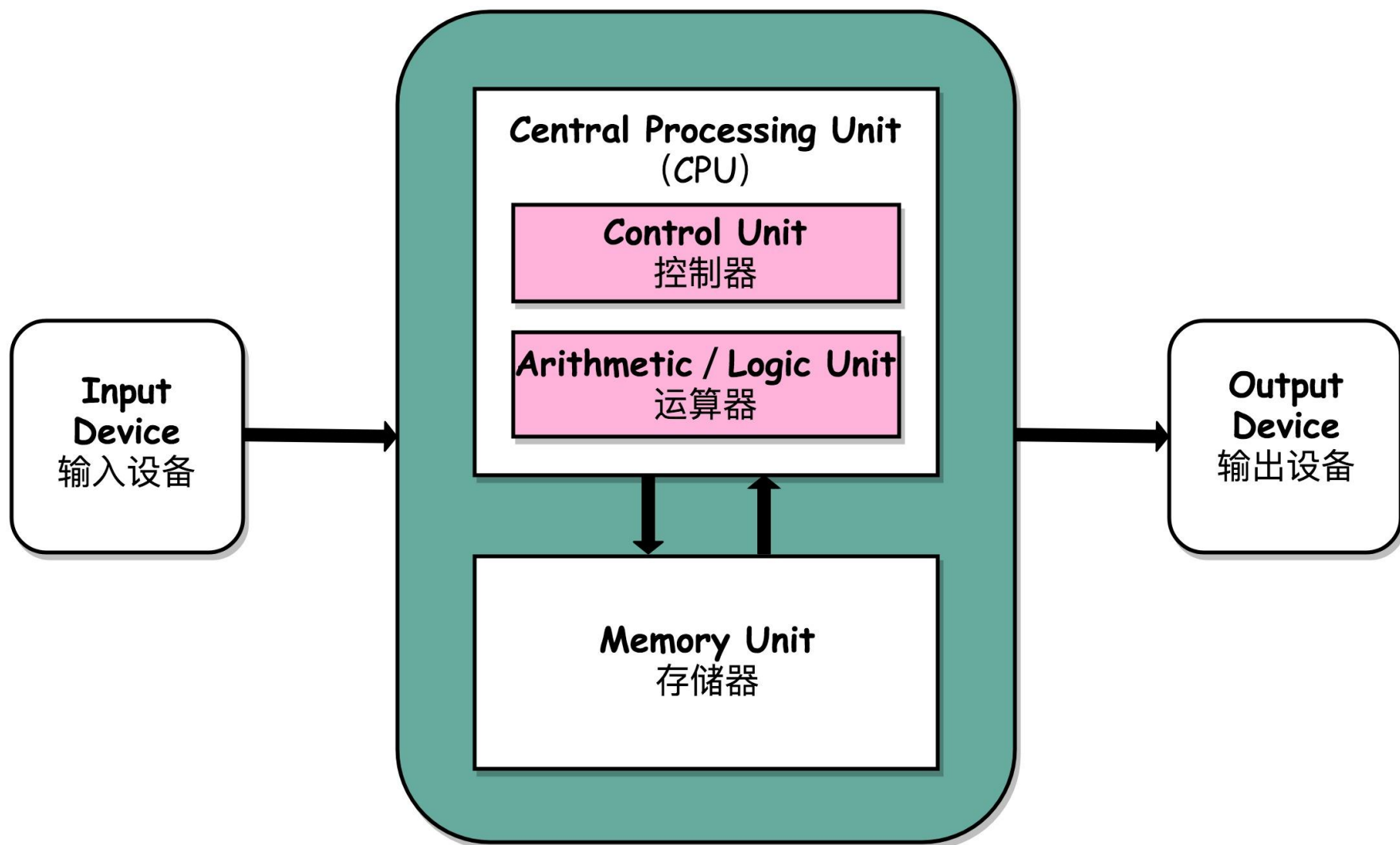


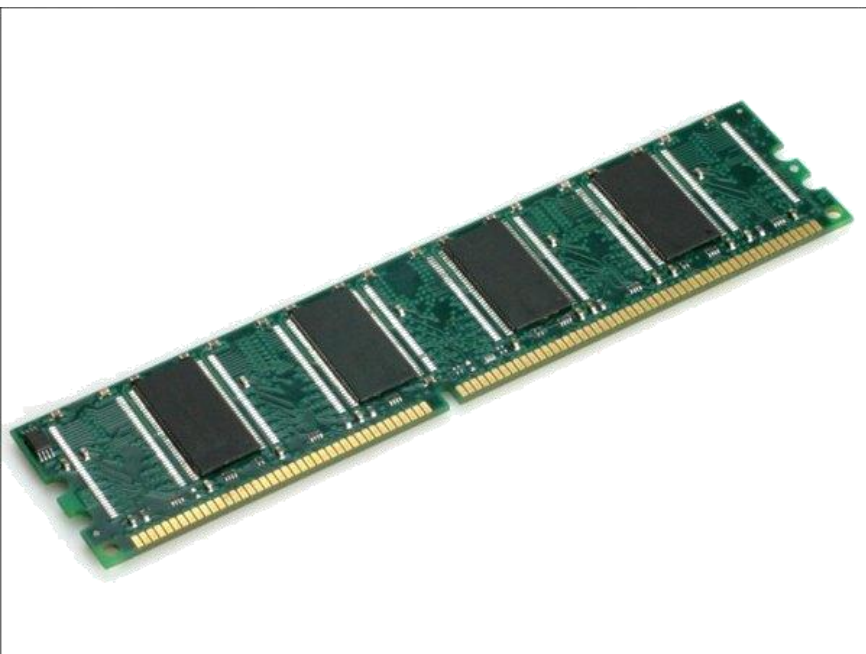
# C++基础： 指针、引用 与动态内存分配

# 冯诺依曼体系下的计算机



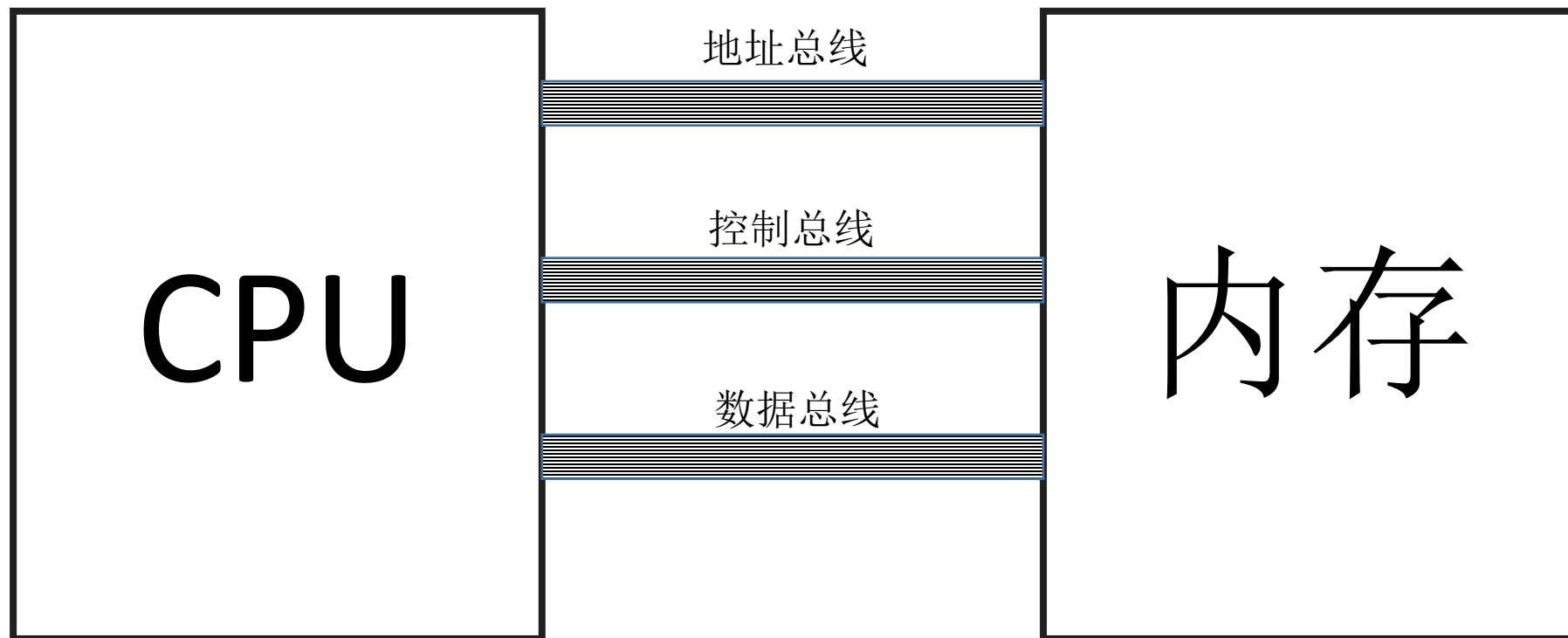
# 内存和CPU

- 物理上的内存和CPU构成了计算机的主要部件。得益于统一的协议和生产标准，当前的大部分主机都通过主板连接内存和CPU并且他们是可更换的。



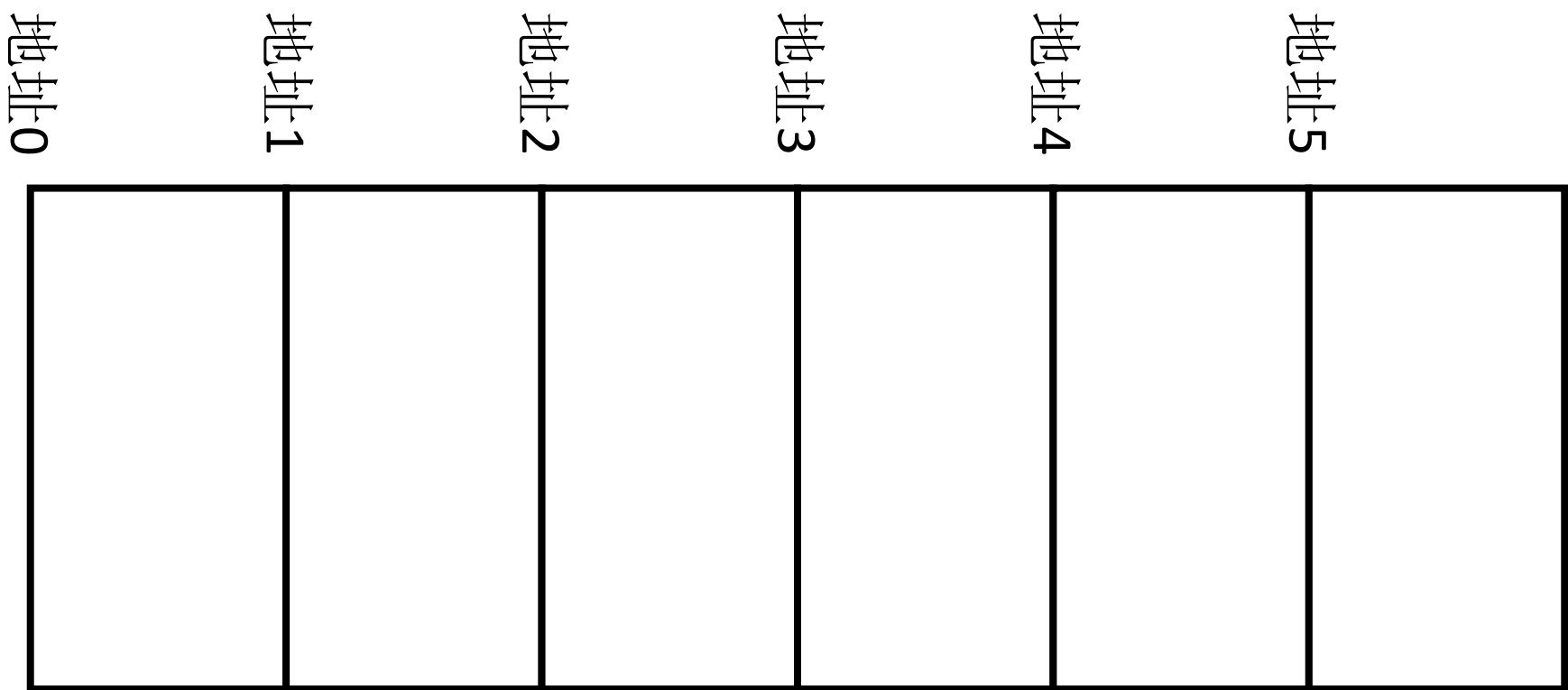
# 内存和CPU

- 逻辑上，CPU和内存通过三个总线连接，分别是地址总线、控制总线 and 数据总线。
- 内存存储了包括指令和数据在内的程序数据，统一存储为二进制编码。CPU通过总线从内存获取一条一条的指令和数据然后执行。



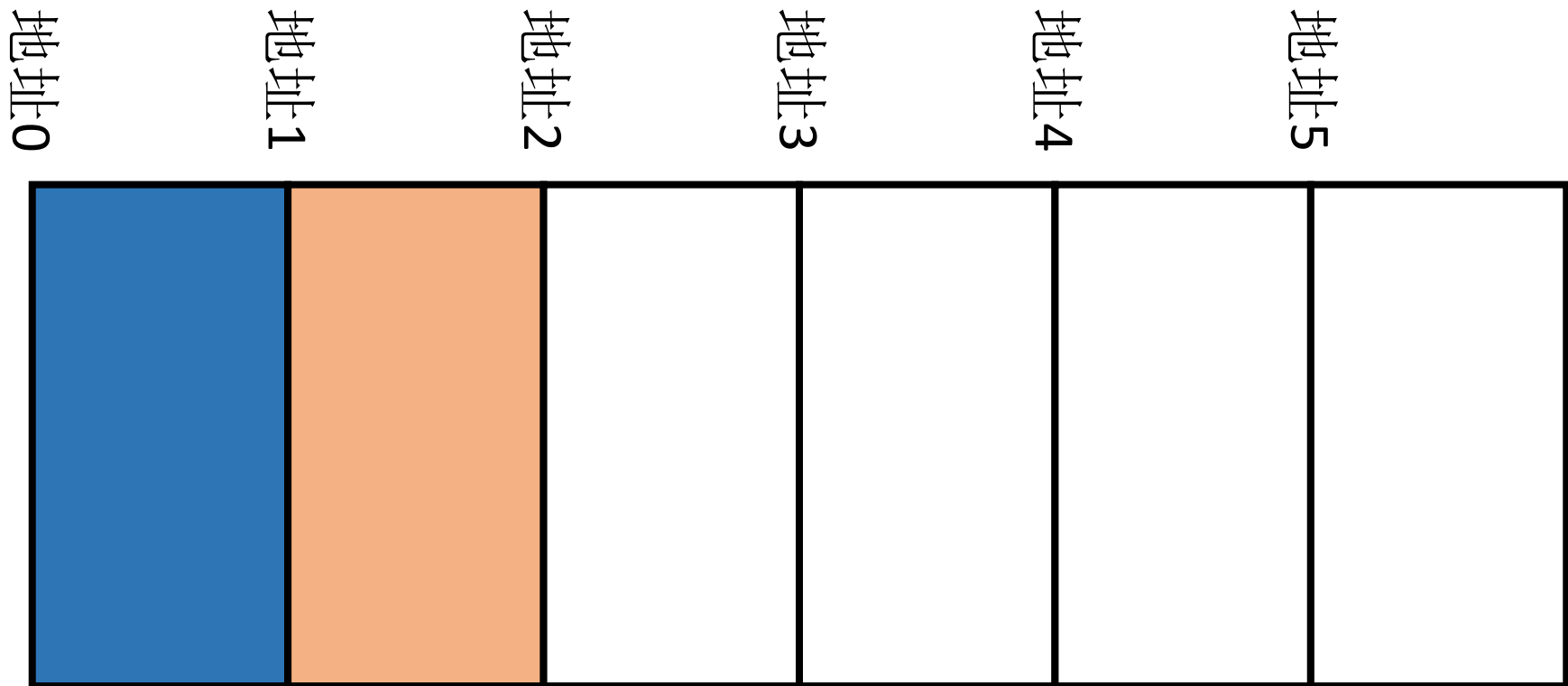
# 内存

- 内存在逻辑上是一段由连续地址编码的连续的空间，存储数据和指令。**CPU**通过地址读取内存里面的数据和指令。



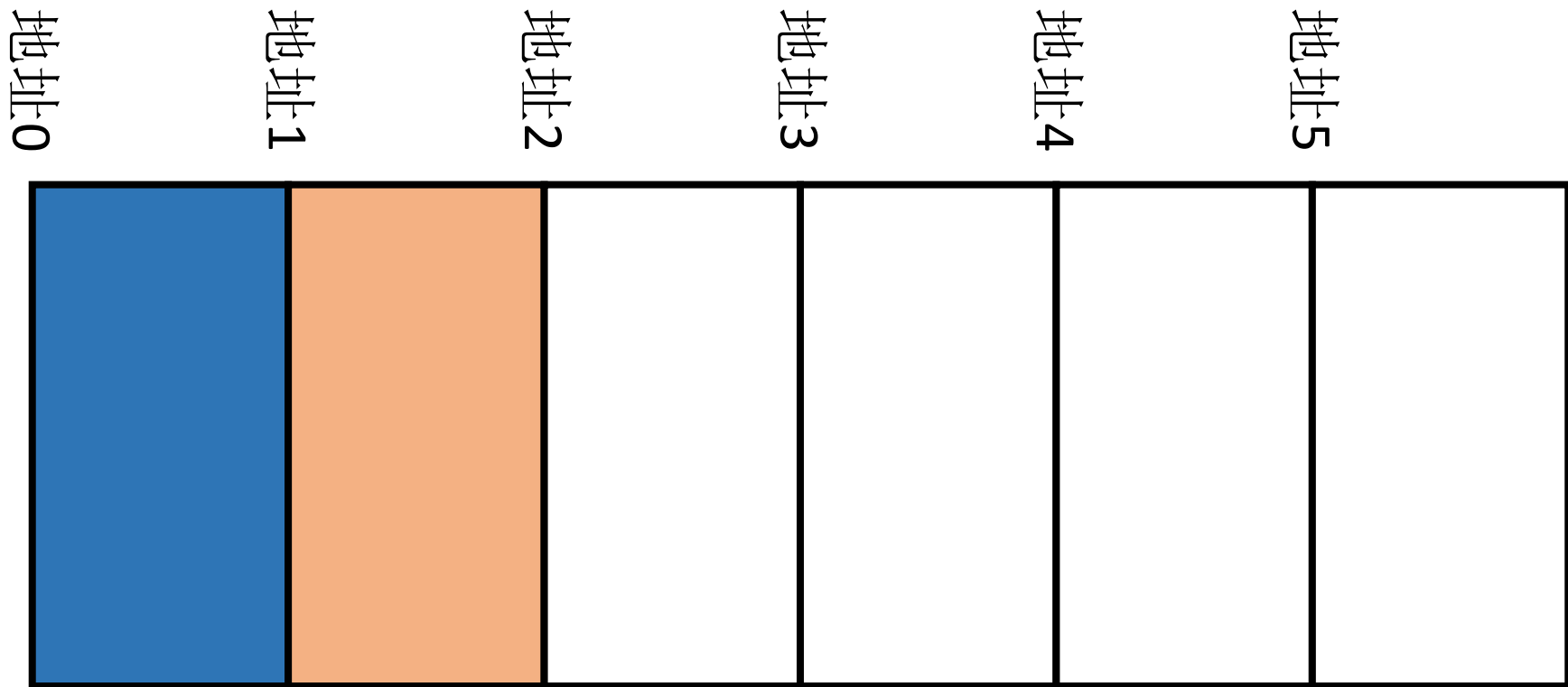
# C++变量与内存

- C++程序中所处理的数据主要以变量（和对象）的形式出现。每个变量有两个最主要的属性，那就是变量的内容和地址。
- 变量对应于内存中存储的数据，而各种运算和控制则对应内存中存储的指令。他们都统一编码存储在内存中



# C++变量与内存

- C++语言也为用户对于变量所占空间的地址提供了进行操作的机制, 这就是指针类型和引用类型。
- 指针和引用不是一种独立的数据类型, 而是一种导出数据类型。



# 值传递中的指针参数

- 函数 `swap1()` 中使用了 `*a` 和 `*b`，这里符号 “`*`” 不是乘法运算符，而是表示指针变量（`a` 或 `b`）所指向的那个变量。
- 在 `swap1()` 中有两个指针型参数，属于赋值形参，在调用时创建两个新的指针变量 `a` 和 `b`，把实参表达式计算的结果地址赋给 `a` 和 `b`，从而使指针 `a` 和 `b` 指向相应的变量。

```
void swap1(float *a, float *b){                                //交换两个变量的值
    float temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```



# 目录

## 单链表



## 双向链表



## 循环链表



## • 指针类型

- 指针变量说明
- 指针变量的操作
- 指针的关系运算
- 指针与数组
- 字符串指针
- 指针与函数

## • 指针与动态内存分配

- 动态分配运算符
- 用指针进行内存动态分配

## • 引用类型

- 引用变量的说明
- 引用和指针的比较
- 引用型参数
- 引用型的函数返回值

# 指针变量说明

- 指针（Pointer）类型的变量说明格式为：  
    <类型名>\*<指针变量名>=<初值>;
- **类型名**：任一基本类型名，基本类型的派生类型名，用户定义的类型、枚举类型、结构及联合类型名。
  - 类型名为 `void` 时，称为不确定类型的指针类型。
  - 类型名也可以是由<类型名>\*表示的指针类型名，这时称为多级指针（指针类型的指针）。
- **符号\***：表示其后说明的变量为指针变量（这里的“\*”不是运算符）。
- **指针变量名**：标识符。
- **初值**：可缺省。它可以是该类型的某一变量的地址。
- **注意**：指针本身也是一个变量，需要内存存储，也有地址和长度。

# 指针变量说明

- ptr 是一个未赋初值的 int 型的指针变量。
- pa 是 float 型的指针变量，被赋初值为&a，即 pa 当前的值为变量 a 的地址，或说 pa 指向变量 a。如把&a 赋给 ptr 是不合法的，因为后者是整型指针。
- p[] 是浮点型指针数组，它由两个指针元素组成，分别被赋值&a、&b。
- pn 是一个被赋初值为 NULL 的指针变量，规则规定 NULL 与整数 0 通用，它是惟一可以赋给任一类型指针变量的值，表示当前该指针未指向任一变量。

```
int *ptr;
```

```
float a=3.0, b, c[4]; float * pa=&a;
```

```
float *p[2]={&a, &b};
```

```
float *pn=NULL;
```

# 指针变量说明

因此，一个指针变量，可有 3 种状态：

- 未赋任何值，“悬空”状态。
  - 被赋予 NULL 值，未指向任一变量。
  - 指向某一变量的地址。
- pc 是一个浮点型指针变量，c 是指向数组的首元的指针，这时把 c 的值赋给 pc，意味着 pc 的值为数组 c[ ] 的首元素 c[0] 的地址。
  - pf 也是浮点型指针变量，初值是通过动态内存分配运算符 new 的“运算”结果，生成一个无名的 float 型变量，其地址赋给了 pf。
  - pp 是一个对象指针，假定 point 是一个已定义类名，pp 存放一个 point 类对象的地址，或说可用 pp 指向 point 类的一个对象

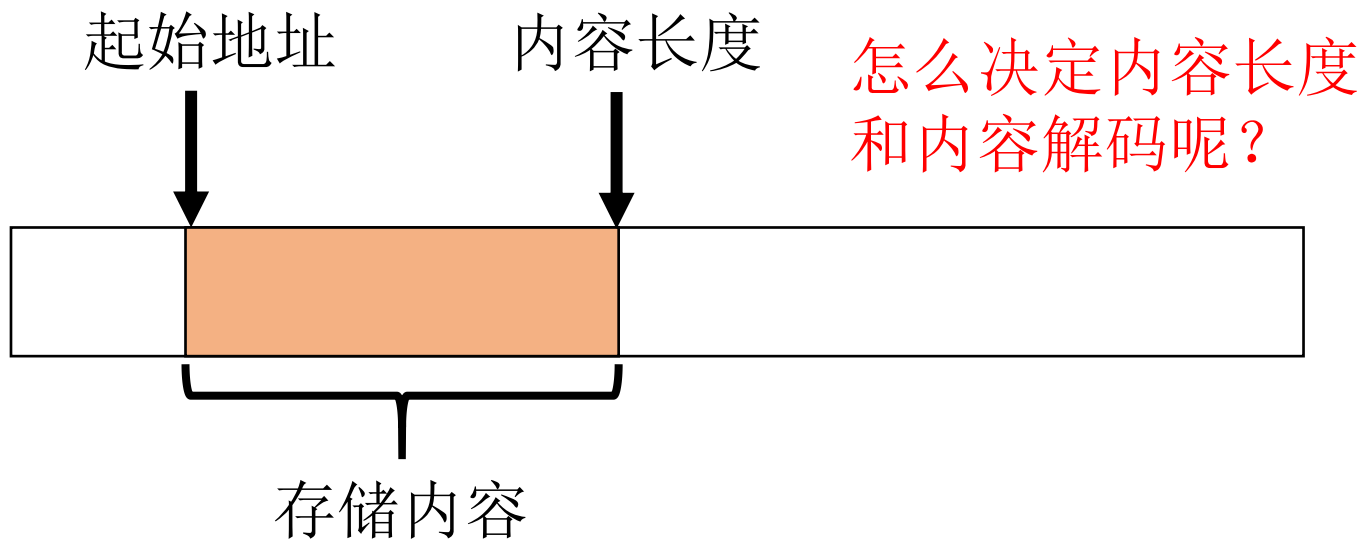
```
float *pc=c;
```

```
float *pf=new float;
```

```
point *pp;
```

# 指针变量说明

- 指针指向了变量的起始地址，变量类型则决定了从起始地址开始变量的内容长度和解码格式。
- 那么变量类型分别占用多少内存空间呢？又是怎么解码存储空间的内容（二进制串）的呢？（参见变量那一章的内容）



# 指针变量的操作

- 取地址运算&和取内容运算\*

- 取地址运算为一单目运算，运算符为&，例如：

```
int a=3,*pa=&a;
```

- 其中&a 表示变量 a 的地址，它可以作为指针 pa 的值。

- 使用&时应注意：

- 其后只可用一个变量名，而不可用字面常量或一般表达式。(为什么?)
  - 赋值时，pa 与 a 的类型应一致。

- 取内容运算为一单目运算，运算符为\*，例如：

```
int a=3,*pa=&a;
```

```
*pa=5;
```

```
cout<<a<<*pa;
```

# 指针变量的操作

```
int a=3,*pa=&a;
```

```
*pa=5;
```

```
cout<<a<<*pa;
```

- `*pa=5;` 和 `cout<<a<<*pa;` 中的 `*pa` 表示指针 `pa` 当前所指向的变量 `a`，因此，`*pa=5;` 相当于 `a=5;`，这里的 `*pa` 被称为“左值”，相当于变量 `a` 占用的空间，而后一个 `*pa` 则相当于变量 `a` 的值，故二者显示出的值应都是 5。第一行中的 `*pa` 要理解为说明 `pa` 是“`int*`”类型的变量。
- 特别注意：`int* p` 表示指针类型申明，但是 `*` 需要跟着 `p` 走而不是 `int` 走，所以我们不能说 `int*` 表示 `int` 指针导出类型，如上的 `int a=0, *p=NULL;`

# 指针变量的操作

- 如果指针变量是指向数组的，例如：

```
int a[20];  
int *pa=a;
```

- 则指针 **pa** 可以有限制地进行+、-运算或增量（++）减量（--）运算。这时，可以用 3 种方式表示该数组的元素：

- 下标变量的形式：**a[0], a[1], a[2], a[3], ..., a[19]**。
- 用数组名 **a**（一个常量指针）：**\*a, \*(a+1), \*(a+2), \*(a+3), ...**
- 用指针变量 **pa**（=a）：**\*pa, \*(pa+1), \*(pa+2), \*(pa+3), ...**

- 然 3 种方式都可以，不过应注意的是，这里 **pa** 是变量，它可能是变化的，例如：

```
pa+=3;
```

- 执行后，**pa** 已指向 **a[3]**，即 **\*pa** 为 **a[3]**。

```
pa--;
```

- 执行后，**pa** 已是指向 **a[2]**。



# 指针变量的操作

- 当数据为多维数组时，上述对应关系为：

```
int a[2][3];
```

```
int * pa = a[0]; //或 int * pa = &a[0][0];
```

- 注意：在二维数组 `a` 中，包括两个一维数组分量，其中，一维数组 `a[0]` 等同于 `&a[0][0]`，所以，`*a[0]` 等同于 `*(&a[0][0])` 即 `a[0][0]`。
- 可用 3 种方式表示其 6 个元素：
  - `a[0][0]`、`a[0][1]`、`a[0][2]`、`a[1][0]`、`a[1][1]`、`a[1][2]`。
  - `*a[0]`、`*(a[0] + 1)`、`*(a[0] + 2)`、`*(a[0] + 3)`、`*(a[0] + 4)`、`*(a[0] + 5)`。
  - `*pa`、`*(pa + 1)`、`*(pa + 2)`、`*(pa + 3)`、`*(pa + 4)`、`*(pa + 5)`。
- 注意：
  - 要深刻理解指针加减的单位；
  - 要深刻理解多维数组在内存中存储形式；
  - 要深刻理解指针的指针，也就是二维数组的指针是什么意思。

# 指针的关系运算

- 指针变量可以参加关系运算。这要分 3 种情况：
  - 一般指针可以进行相等和不等的比较，指向同一变量（地址）者为相等，否则不等。
  - 任一指针可以和指针常量 NULL 进行相等和不等的比较。如一个指针 **p** 已经指向了某变量，则它不等于 **NULL**。
  - 数组指针可以指向该数组的各个元素，且一个数组的各个元素在内存中是顺序存放的，故数组指针之间不但可以进行相等和不等比较，也可以进行大于、小于、大于等于、小于等于等的比较。
- 指针和所有类型一样，也可进行赋值运算。

```
float  a, b, c[10];
```

```
float  *p1=&a, *p2=NULL, *p3=c, *p4;
```

```
if (p1 != p2) p2=p3+2;
```

# 指针与数组

- 指针与数组都是导出类型，它们都是在其他类型的基础上被定义的。那么它们互相之间可否“导出”呢？（类似于结构和数组）
- 指向数组元素的指针
- 前面所讲的数组指针，实际上应更确切地称之为指向数组元素的指针，例如：

```
int n[10];
```

```
int *pn=n;
```

- 则指针 `pn` 指向数组 `n` 的首元 `n[0]`。在这里 `pn=n` 与 `pn=&n[0]` 起相同的效果，因为数组名就是一个指向其首元的一个指针常量。

# 指针与数组

- 指向数组的指针
- 把数组作为整体，称指向这样一个整体的指针为指向数组的指针，对于这类指针，其说明语句格式为：  
    <类型名>(\*<指针变量名>)[<数组元素数>];
- 例如：  
    float A[2][4];  
    float (\*pa) [4]; float \*p=&A[0][0];  
    pa=A;
- 这时指针 pa 指向一维数组 A[0]（A[0]有 4 个元素），执行增量运算 pa++; 后，pa 指向一维数组 A[1]，数组 A[1]的 4 个元素。
- 指针 p 和指针 pa 指向的地址是一样的，但 p 是指向一个整型下标变量 A[0][0]，而 pa 则是指向整型一维数组 A[0]，前者占内存空间可能是 4 bytes，而后者则占 16 bytes。
- 从指向数组指针和指向数组元素指针的区别可以看到：虽然指针变量的内容都是地址，但指针变量的类型（指向的对象类型）则决定了该地址指向的存储区的大小。

# 指针与数组

- 指针数组
- 由指针组成的数组，常常在程序中出现，其说明格式为：  
    <类型名>\*<数组名>[<元素数>];
- n 维指针数组为：  
    <类型名>\*<数组名>[<元素数 1>][<元素数 2>]...[<元素数 n>];
- 直观的区别是：
  - 它比一般数组增加了符号“\*”。
  - 它比指向数组的指针少了一对括号“（）”。
- 例如：

```
int a, b, c, d, A[2][4];  
int *p1[4]={&a,&b,&c,&d};  
int *p2[2]={A[0],A[1]};
```

# 指针与数组

- 下面定义的指针展示了指向数组元素的指针、指向数组的指针、指针数组的区别：  
    `int *p;`  
    `int *p1[2];`  
    `int (*p2) [4];`
- `p` 是指向 `int` 型变量的指针，`p1` 是由两个整型指针组成的指针数组，而 `p2` 则是一个指向一维整型数组的指针。

# 字符串指针

- 字符串是一种特殊的数据形式，在数组一节，我们已经指出字符串和字符数组的关系，即字符串就是数组加串尾符。
- 字符串指针的说明语句的格式与字符类型指针相同，例如：  

```
char ch= 'a'; char *pc1=&ch; char *pc2="world";  
char as[10]="Chinese"; char *pc3=as;
```
- 变量 `ch` 是字符型，它被赋初值字符 `'a'`。
- 变量 `pc1` 是字符指针，它被赋初值为字符变量的地址。这时 `pc1` 是一个字符型指针。
- 变量 `pc2` 和 `pc1` 一样说明为字符指针，但它被赋初值为字符串常量 `“world”`，它的内容是字符串第一个字符 `'w'` 的存储地址。
- 数组 `as[10]` 是一个长度为 10 的字符数组。为 `as` 所赋的初值为字符串常量 `“Chinese”`，这个常量在数组中占 8 个分量的位置，包括 7 个字母和一个串尾符 `“\0”`。
- 变量 `pc3` 说明为字符指针，并令其指向字符数组 `as[]` 的首元，这时，`pc3` 也是一个字符串指针。

# 字符串指针

- 字符串指针可以按一般字符指针的规则工作，例如：

```
while (*pc2 != '\0')  
    cout<<*pc2++;
```

- 执行上述 while 语句将在屏幕上显示 world。类似地：

```
while (*pc3 != '\0')  
    cout<<*pc3++;
```

- 执行的结果是显示 Chinese。
- 字符串指针有时可以按不同于一般指针变量的方式操作，即有时它可以表示字符串整体，而不是单单表示当前指向的那个字符。

| 定义方式                                                       | 是否自动加 <code>'\0'</code>           | 正确性    |
|------------------------------------------------------------|-----------------------------------|--------|
| <code>char str[] = "hello";</code>                         | ✅ 自动添加 <code>'\0'</code>          | 推荐 ✅   |
| <code>char str[10] = "hello";</code>                       | ✅ 自动添加 <code>'\0'</code> ，并填充剩余部分 | 推荐 ✅   |
| <code>char str[] = {'h', 'e', 'l', 'l', 'o'};</code>       | ❌ 不会自动加 <code>'\0'</code>         | 可能出错 ❌ |
| <code>char str[] = {'h', 'e', 'l', 'l', 'o', '\0'};</code> | ✅ 手动添加 <code>'\0'</code>          | 正确 ✅   |



# 字符串指针

- 把字符串作为字符数组，然后按字符依次进行输入输出是可行的，但比较麻烦。因此，C++提供了整体输入输出的方法。
- 把字符串变量用字符数组或字符串指针的形式表示，然后直接进行 I/O 操作：  

```
cin>><字符串>;  
cout<<<字符串>;
```
- 这里的字符串可以用字符串指针 pc2、pc3 或字符数组名 as 来表示，例如：  

```
char as [10]="world"; char *pc2="world"; char *pc3=as;  
cin>>as; cin>>pc2; cin>>pc3;  
cout<<pc2; cout<<pc3; cout<<"world";
```

# 字符串指针

- 以字符串指针组成的数组使用起来比较方便，例如：

```
char *menu [ ] = { "File", "Edit", "Search", "Help" };
```

- 就是一个字符串指针数组，它的每个元素指向一个字符串常量。于是 menu[0]、menu[1]、menu[2]、menu[3]将分别表示"File"、"Edit"、"Search"、"Help"。
- 字符串指针数组就是main函数默认参数的数组。

# 字符串指针

- 利用字符串指针的实现字符串功能的标准函数
- 求字符串长度。

```
unsigned strlen (const char *str);
```

- 返回字符串 `str` 的长度（不包括串尾符）。
- 字符串拷贝。

```
char *strcpy (char *str1 , char *str2);
```

- 把字符串 `str2` 拷贝给字符串 `str1`（覆盖原来内容），返回 `str1` 的地址。
- 字符串连接。

```
char *strcat (char *str1 , char *str2);
```

- 把 `str2` 连接到字符串 `str1` 的后面（使 `str1` 加长），返回 `str1` 的地址。
- 字符串比较。

```
int strcmp (char *str1 , char *str2);
```

- 此函数把 ASCII 码表作为字符表，按字典序比较两个字符串 `str1` 和 `str2`。

# 指针与函数

- 在冯诺依曼体系结构下的计算机，指令和数据都被统一编码为二进制串存储于内存中，因此函数调用指令也有其调用地址和返回地址，可以用指针指向函数调用。
- 指针与函数的关系密切，主要分 3 部分：**指针作函数参数、作函数返回类型和指向函数的指针。**

# 指针与函数-指针作函数参数

- 指针是一种特殊类型的数据，它的值是另外某种类型的变量的地址。指针作函数的参数可以起到其他类型参数所起不到的作用。
- 函数的参数（引用型参数除外）在调用过程中，需把实参（表达式）的值赋给仅在调用过程中有意义的形参，参加到函数体的运算之中。这样的机制很难实现对相对于函数的外部的“全局”变量作某些处理。
- 传递指针可以通过参数传进了要改变内容的全局变量的地址，从而达到以参数形式输入，又能改变函数外变量内容的目的。

```
void swap (float x, float y) {  
    float t=x;  
    x=y;  
    y=t;  
};  
  
void swap (float *pa, float *pb) {  
    float t=*pa;  
    *pa=*pb;  
    *pb=t;  
};
```

# 指针与函数-函数返回指针

- 返回值为指针的函数称为指针型函数，其返回类型的说明应在指明指针的对象类型后加“\*”符号。
- 指针型函数的设计，**应注意返回的指针在该函数被调用的域内是有确切的对象变量的。切不可返回指向函数内说明的局部变量或参数变量的指针或无指向的指针。**
- **注意：和返回引用类型一样，返回的指针指向的不应该是临时变量或者函数里的局部变量，不然一返回地址，该地址的变量就被释放了，那这个返回的指针就成了没人要的野指针。**

```
char *menu[ ] = {"Error!", "File", "Edit", "Search", "Help"};
char *menuitem (int m) {
    return (m < 1 || m > 4) ? menu[0] : menu[m];
};
```

# 指针与函数-函数指针

- 函数不是数据，但它与变量还是有两点相通之处：一个是它有类型（返回类型），另一个是它也有地址，称为入口地址。
- 函数的地址也可作指针的值，这就是函数指针。
- 函数指针的说明格式与函数的原型相似，主要区别是：原来的“<函数名>”，用“(\*<函数指针名>)”所代替，例如：

```
int (*pf) (float);
```

- 其中 pf 是一个函数指针变量，由于对 pf 的说明中已规定了**函数的返回类型**（有时还包括**存储类型**）和**参数表**，因此，指针 pf 只能指向这类函数。

```
int f1 (float); int (*pf) (float)=&f1;
```

# 指针与函数-函数指针

- C++语言本身不允许把函数作为参数，然而有了函数指针，就可以通过函数指针把函数作为参数使用。

```
float simpson (float a, float b, float (*pf)(float));  
float a=3.2, b=4.2;  
float f1 (float){... }; //函数体略  
float f2(float){... }; //函数体略  
float (*pf1) (float)=&f1; //函数指针 pf1 指向函数 f1  
cout<<simpson (a, b, pf1) <<endl;  
a=4.0; b=10.5;  
pf1=&f2; //pf1 指向函数 f2  
cout<<simpson (a, b, pf1) <<endl;
```

- 两次调用分别计算两个不同函数 f1 和 f2 的定积分值。



# 目录

## 单链表



## 双向链表



## 循环链表



## • 指针类型

- 指针变量说明
- 指针变量的操作
- 指针的关系运算
- 指针与数组
- 字符串指针
- 指针与函数

## • 指针与动态内存分配

- 动态分配运算符
- 用指针进行内存动态分配

## • 引用类型

- 引用变量的说明
- 引用和指针的比较
- 引用型参数
- 引用型的函数返回值

# 动态分配运算符

- **new** 运算是程序中除了变量说明之外，又一种生成变量的方法，不过用 **new** 生成的变量为**无名动态变量**，它返回的是一个该类型的指针值，程序通过指针对这个变量进行操作。

```
int *pi, a=5;
```

```
char *pc;
```

```
float *pf, f=3.0;
```

```
pi=new int;
```

//生成一动态 int 类型变量

```
pc=new char[4];
```

//生成动态 char 类型数组

```
pf=new float (4.7);
```

//生成动态 float 类型变量且赋初值

```
*pi = a*a;
```

//动态变量\*pi 被赋值

```
for (int i=0; i<4; i++)
```

//动态 char 型数组的每个分量被赋值

```
    *(pc+i)= 'a';
```

```
f+=*pf;
```

//动态变量\*pf 参加运算

# 动态分配运算符

- 什么是动态变量？

```
int    *pi, a=5;
```

```
char    *pc;
```

```
float    *pf, f=3.0;
```

```
pi=new int;
```

//生成一动态 int 类型变量

```
pc=new char[4];
```

//生成动态 char 类型数组

```
pf=new float (4.7);
```

//生成动态 float 类型变量且赋初值

```
*pi = a*a;
```

//动态变量\*pi 被赋值

```
for (int i=0; i<4; i++)
```

//动态 char 型数组的每个分量被赋值

```
    *(pc+i)= 'a';
```

```
f+=*pf;
```

//动态变量\*pf 参加运算

# 动态分配运算符

- 用 `new` 生成的一个动态变量与用变量说明语句说明的一个变量作比较：
  - 都要指出变量的类型，类型名要放在 `new` 之后。
  - 都可以赋初值，不是用 “=`<初值>`” 的方式而是用括号 “(`<初值>`)” 的方式。
  - 都可以说明为数组，加数组运算符 `[]`。
  - 动态变量没有变量名，需用指针变量接收到它的地址后，通过指针运算符 “`*`” 进行操作。

```
pi=new int;
```

```
//生成一动态 int 类型变量
```

```
pc=new char[4];
```

```
//生成动态 char 类型数组
```

```
pf=new float (4.7);
```

```
//生成动态 float 类型变量且赋初值
```

```
*pi = a*a;
```

```
//动态变量*pi 被赋值
```

# 动态分配运算符

- delete 运算用来撤销或释放由 new 生成的动态变量，例如：  
    delete pi; //释放 pi指向的动态 int 变量  
    delete pf; //释放 pf 指向的动态 float 变量  
    delete [] pc; //释放 pc 指向的动态数组
- 动态变量与一般变量的主要区别就是它可以在程序运行过程中任意被撤销。而一般变量则必须在其所说明的程序块结束时自动撤销。
- 也就是说，动态变量与一般变量的生存期和作用域非常不同。

# 用指针进行内存动态分配

- 动态内存分配指：使用运算符 `new` 和 `delete`，通过指针引用，可以在程序的任何地方进行内存的分配和释放。例如：

```
int *pint; char *pchar; float *pfloat;
```

```
pfloat = new float; pchar = new char; pint = new int;
```

- 这样就生成了 3 个（浮点型、字符型、整型）变量，但它们没有名字。由于 3 个变量的地址 分别存在指针 `pfloat`、`pchar` 和 `pint`，故在程序中使用这 3 个变量时，都通过指针：

```
*pchar = 'A'; *pint = 5; *pfloat = 4.7; *pint++;
```

- 当不再需要指针所指向的这些变量时，可在程序的任何地方释放掉它们：

```
delete pchar; delete pint; delete pfloat;
```

- 注意：这里释放的是动态的（`char`、`int`、`float`）变量，而不是指针变量（`pchar`、`pint`、`pfloat`）。

# 用指针进行内存动态分配

- 在使用动态变量时应注意的是要保护动态变量的地址，比如在释放或者保存前一个动态变量的地址后，在更改指针的值。例如：

```
pi=new int;
```

- 不要轻易地更改指针 `pi` 中的值，如果未保留 `pi` 的值，又执行了 `pi=&a;` 语句之后，那么原来的值就再也找不回来了！！！！这时候再释放 `pi` 就出错了，因为 `a` 不是动态变量，而原来生成的动态变量则面临无法释放的尴尬局面：

```
pi=&a; delete pi;
```

- 如果很多这样的变量不释放，就会导致内存越用越多，最后死机。此外这种未释放的动态变量还可能导致内存泄漏等安全问题。
- C++还保留了从C语言中继承下来的用于动态存储分配的 **malloc()**、**free()** 等标准函数，它们已经可以被 `new` 和 `delete` 运算所取代。

# 目录

## 单链表



## 双向链表



## 循环链表



### • 指针类型

- 指针变量说明
- 指针变量的操作
- 指针的关系运算
- 指针与数组
- 字符串指针
- 指针与函数

### • 指针与动态内存分配

- 动态分配运算符
- 用指针进行内存动态分配

### • 引用类型

- 引用变量的说明
- 引用和指针的比较
- 引用型参数
- 引用型的函数返回值



# 引用变量的说明

- 引用不仅像数组和指针那样依赖于已有的类型，而且它还依赖于一个已有的变量。从直观上说，一个引用变量是一个已定义的变量的别名。
- 引用变量的说明格式与指针变量说明相似：  
    <类型名>&<变量名>=<对象变量名>;
- 与指针说明的区别是：
  - 用符号“&”代替符号“\*”。
  - 赋初值部分不可缺省。（为什么？）

```
int size=5, color;
```

```
int & refs=size;
```

```
int & refc=color;
```

# 引用和指针的比较

- C++语言中增加引用这种数据形式，主要是用于在一定的范围内，代替或改进指针的作用。虽然在说明方式上十分相似，但在概念上却有着明显的不同。其区别最主要有两点：
  - 指针表示的是一个对象变量的地址，而引用则表示一个对象变量的别名。因此在程序中表示其对象变量时，前者要通过取内容运算符\*，而后者可直接代表。
  - 指针是可变的，它可以指向变量 m，也可以指向变量 n，而引用变量只能在定义时一次确定，不可改变。
- 引用类型变量与其他类型变量不同，它没有自己的值和地址空间，只是作为另一个变量的别名，在它的生存期期间两个名字绑定在一起，因此，引用类型的使用是有限制的：
  - 引用类型变量不能被引用。
  - 引用类型不能组成数组。
  - 引用类型不能定义指针。

# 引用型参数

- 引用型参数在函数被调用时，**相应的实参必须是对应类型的变量或对象**；在调用函数体运行前，生成该实参的引用变量；在整个函数体运行过程中，这个引用变量相当于作为实参的变量或对象的别名，直到函数调用结束返回。优点是：
  - 可以把函数外的变量以别名的形式引入到函数体内参加运算，非常方便，这种方式比用指针解决这个问题**更合理**。
  - 不必在调用时创建与实参变量或对象对应的值的参数变量，当实参变量或对象占用内存较多时，可以**节省内存**。
  - 用指针也可以实现类似于引用调用的效果，但由于指针可以改变内容，任意赋值，因此它不如引用型**参数安全**。

# 引用型的函数返回值

- 当把函数的返回类型说明为引用型时，这个函数返回的不仅仅是某一变量或对象的值，而且返回了它的“别名”，该函数的调用也可以被赋值。例如：

```
int & maxr (int & m, int & n) {  
    if (m>n) return m;  
    return n; }  
maxr (a, b)= 10;
```

- 函数的调用本身也可以作为变量和对象来使用

# 目录

## 单链表



## 双向链表



## 循环链表



## • 指针类型

- 指针变量说明
- 指针变量的操作
- 指针的关系运算
- 指针与数组
- 字符串指针
- 指针与函数

## • 指针与动态内存分配

- 动态分配运算符
- 用指针进行内存动态分配

## • 引用类型

- 引用变量的说明
- 引用和指针的比较
- 引用型参数
- 引用型的函数返回值

# 链表的基本概念

- 此前我们已经知道了数组是一种线性的结构，其由相同的元素组成的一系列的值。
- 与之对应的链表是一种线性数据结构，它由一系列节点构成，每个节点包含数据和一个指向下一个节点的指针。
- 链表中的节点可以在内存中任意位置，而不像数组一样需要连续的内存空间。

# 链表节点的结构

- 链表中的每个节点由两个部分组成：一个是数据域，用于存储节点的数据，另一个是指针域，用于存储下一个节点的地址。链表节点的结构可以用C++代码表示为：

```
template<typename T>
struct ListNode {
    T val; // 数据域
    ListNode<T> *next; // 指针域，指向下一个节点
    ListNode(T x) : val(x), next(NULL) {} // 构造函数
};
```

- 其中，T是节点存储的数据类型，val表示节点存储的数据，next是指向下一个节点的指针，NULL表示空指针。

# 链表的问题

- 那么链表的数据内容必须一致吗？
- 链表的地址空间必须连续或者不连续的吗？
- 链表相比于数组的优势和劣势分别是什么？



# 单向链表

- 单向链表，指的是有开始和结束位置，并且只有一个指向后继或者前驱指针的链表。
- 单向链表的基本操作包括遍历、插入、删除和反转等操作。
- 链表的遍历操作是指依次访问链表中的每个节点，可以使用一个指针从头节点开始依次访问下一个节点，直到指针为空（即遍历到了链表的末尾）。链表的遍历操作的时间复杂度为 $O(n)$ ，其中 $n$ 是链表的长度。

```
template<typename T>
void traverseList(ListNode<T> *head) {
    ListNode<T> *p = head;
    while (p != NULL) {
        cout << p->val << " ";
        p = p->next;
    }
    cout << endl;
}
```

# 单向链表

- 链表的插入操作是指在链表中插入一个新节点，可以在任意位置进行插入操作。链表的删除操作是指从链表中删除一个节点，同样可以在任意位置进行删除操作。

```
struct ListNode {  
    int val;  
    ListNode* next;  
    ListNode(int x) : val(x), next(NULL) {}  
};
```

```
ListNode* insertAtHead(ListNode* head, int val) {  
    ListNode* newNode = new ListNode(val);  
    newNode->next = head;  
    return newNode;  
}
```

```
ListNode* deleteNode(ListNode* head, int val) {  
    if (head == NULL) {  
        return NULL;  
    }  
    if (head->val == val) {  
        return head->next;  
    }  
    ListNode* cur = head;  
    while (cur->next != NULL && cur->next->val != val) {  
        cur = cur->next;  
    }  
    if (cur->next != NULL) {  
        cur->next = cur->next->next;  
    }  
    return head;  
}
```

# 双向链表

- 双向链表是一种每个节点都有前驱和后继指针的链表，相比单向链表，双向链表可以双向遍历，删除节点时无需遍历查找前驱节点。双向链表的节点结构通常包括当前节点的数据域、指向前驱节点的指针prev和指向后继节点的指针next。

```
struct ListNode {  
    int val;  
    ListNode *prev;  
    ListNode *next;  
    ListNode(int x) : val(x), prev(nullptr), next(nullptr) {}  
};
```

- 双向链表的操作与单向链表类似，但需要同时维护前驱和后继指针。

# 双向链表

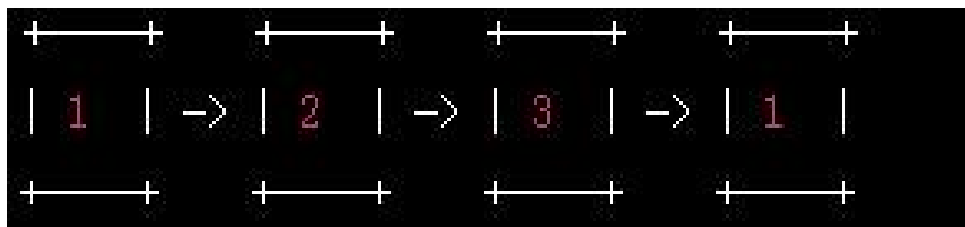
- 双向链表的插入和删除操作与单向链表类似，但需要同时维护前驱和后继指针。

```
// 在当前节点后插入一个节点
void insert(ListNode* cur, int val) {
    ListNode* newNode = new ListNode(val);
    newNode->prev = cur;
    newNode->next = cur->next;
    cur->next->prev = newNode;
    cur->next = newNode;
}

// 删除当前节点
void remove(ListNode* cur) {
    cur->prev->next = cur->next;
    cur->next->prev = cur->prev;
    delete cur;
}
```

# 循环链表

- 循环链表是一种特殊的链表，它的尾节点指向头节点，形成一个环。循环链表可以用于解决某些特殊问题，如环形缓冲区。



# 思考题

- 什么是地址，什么是地址中的内容，两者的区别是什么？
- 运算符“&”和“\*”各有几种用法？各自的使用方式以及使用含义是什么？
- 什么是指针？地址和指针有什么样的关系？
- 指针的值和类型是怎样规定的？指针的值与其他类型变量的值是否有区别？
- 指针可以进行哪些运算？和普通数据类型的运算有什么不同？
- `void main (int argc, char * argv[]) {...}` 如何细致地描述参数 `argv` 所表达的数据结构？
- 试述指针在函数的参数传递中的作用及其使用方法。指针参数与数组参数是否有某种形式的关联？
- 什么是引用？引用和指针的区别是什么？引用型参数具有哪些优点？
- 引用型的函数返回值与非引用型的函数返回值是否有某些不相同？